SPEC — Task Management API on AWS (Terraform)

> **Versión:** 1.0
> **Fecha:** 2026-04-21
> **Objetivo:** Documento de especificación completo para ser pasado directamente a un agente IA que ejecutará el despliegue de infraestructura en AWS usando exclusivamente Terraform.

---

1. Contexto de Ejecución

- El operador trabaja desde un **container Docker** con AWS CLI y Terraform instalados.
- Las credenciales de AWS se inyectan en el container (variables de entorno o volumen `~/.aws`).
- **No hay acceso a consola web de AWS** durante el despliegue.
- Todo debe funcionar con comandos `terraform` ejecutados dentro del container.
- La solución es **100% Terraform** — sin CDK, CloudFormation, ni scripts de shell adicionales.
- Región por defecto: `us-east-1` (configurable por variable).
- Terraform versión requerida: `>= 1.5`.
- Provider AWS versión: `~> 5.0`.

---

2. Arquitectura General

```
Actor (usuario)
    |
    ▼
ALB (Application Load Balancer) — público, subnets públicas
    |  HTTP :80 / HTTPS :443
    ▼
API REST (ECS Fargate) — subnet privada
    |  Endpoints:
    |    GET    /tasks
    |    POST   /asimiento
    |    PUT    /asimiento
    |    DELETE /asimiento
    |    GET    /tasks/{taskId}
    |    GET    /health
    |
```

```
    ├──────────────────────────┐
    ▼                          ▼
RabbitMQ (Amazon MQ)          PostgreSQL DB
    |  AMQP :5672                  (RDS o EC2+Docker)
    |
    ├──► Consumer Pool      (ECS Fargate, desired=2)
    ├──► Consumer Delete     (ECS Fargate, desired=1)
    └──► Update Task         (ECS Fargate, desired=1)
```

---

3. Decisiones de Diseño

Componente	Decisión	Justificación
Cómputo API	ECS Fargate	Sin gestión de servidores, escalable
Cómputo Workers	ECS Fargate	Misma plataforma, despliegue uniforme
Message Broker	Amazon MQ (RabbitMQ engine)	RabbitMQ gestionado, sin EC2 adicional
Base de datos	RDS PostgreSQL 15 (opción A)	Gestionado, backups, Multi-AZ fácil
	EC2 + Docker PostgreSQL (opción B)	Alternativa si se prefiere control total
Imágenes Docker	Amazon ECR	Registro privado dentro de AWS
Balanceo	ALB (Application Load Balancer)	Layer 7, path-based routing, health checks
Networking	VPC custom con subnets públicas/privadas	Aislamiento, NAT para salida a internet

Opción recomendada para DB: RDS PostgreSQL (Opción A). Usar Opción B solo si se requiere acceso directo al motor o reducción de costos en entornos muy controlados.

---

4. Estructura del Proyecto Terraform

```
/infra
├── environments/
|   └── dev/
```

```
│       ├── main.tf            # Instancia todos los módulos
│       ├── variables.tf       # Variables del entorno
│       ├── outputs.tf         # Outputs finales
│       └── terraform.tfvars   # Valores (sin secrets)
├── modules/
│   ├── networking/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── security-groups/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── alb/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── amazonmq/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── rds/                   # Opción A
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── ec2-postgres/          # Opción B (alternativa a rds/)
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── ecr/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   └── ecs/
│       ├── main.tf            # Cluster + API + Consumers
│       ├── variables.tf
│       └── outputs.tf
└── providers.tf               # Provider AWS + backend config
```

---

5. Módulo: Networking

Archivo: `modules/networking/`

Recursos requeridos

- `aws_vpc` — CIDR `10.0.0.0/16`, DNS hostnames habilitado
- `aws_subnet` × 2 — Públicas en `us-east-1a` y `us-east-1b`, CIDRs `10.0.1.0/24` y `10.0.2.0/24`
- `aws_subnet` × 2 — Privadas en `us-east-1a` y `us-east-1b`, CIDRs `10.0.10.0/24` y `10.0.11.0/24`
- `aws_internet_gateway` — Adjunto a la VPC
- `aws_eip` — Para el NAT Gateway
- `aws_nat_gateway` — En subnet pública `us-east-1a`
- `aws_route_table` × 2 — Una pública (→ IGW), una privada (→ NAT)
- `aws_route_table_association` × 4 — Asociación de cada subnet a su route table

Outputs

```hcl
output "vpc_id"              { value = aws_vpc.main.id }
output "public_subnet_ids"   { value = [aws_subnet.public_a.id, aws_subnet.public_b
output "private_subnet_ids"  { value = [aws_subnet.private_a.id, aws_subnet.private_
```

---

6. Módulo: Security Groups

Archivo: `modules/security-groups/`

Grupos requeridos

SG Name	Inbound	Outbound
`sg_alb`	80 desde `0.0.0.0/0`, 443 desde `0.0.0.0/0`	All
`sg_api`	8080 desde `sg_alb`	All
`sg_rabbitmq`	5672 desde `sg_api` y `sg_consumers`, 15672 desde `sg_api`	All
`sg_consumers`	desde `sg_rabbitmq`	All
`sg_rds`	5432 desde `sg_api` y `sg_consumers`	All

Outputs

```hcl
output "sg_alb_id"       { value = aws_security_group.alb.id }
output "sg_api_id"       { value = aws_security_group.api.id }
output "sg_rabbitmq_id"  { value = aws_security_group.rabbitmq.id }
output "sg_consumers_id" { value = aws_security_group.consumers.id }
output "sg_rds_id"       { value = aws_security_group.rds.id }
```

---

7. Módulo: ALB

Archivo: `modules/alb/`

Recursos requeridos

- `aws_lb` — Tipo `application`, esquema `internet-facing`, en subnets públicas, con `sg_alb`
- `aws_lb_target_group` — Tipo `ip` (requerido para Fargate), puerto `8080`, protocolo `HTTP`, VPC id
  - Health check: `GET /health`, código esperado `200`, intervalo `30s`, threshold `2`
- `aws_lb_listener` — Puerto `80`, protocolo `HTTP`, acción forward al target group

Outputs

```hcl
output "alb_dns_name"       { value = aws_lb.main.dns_name }
output "target_group_arn"   { value = aws_lb_target_group.api.arn }
output "listener_arn"       { value = aws_lb_listener.http.arn }
```

---

8. Módulo: Amazon MQ (RabbitMQ)

Archivo: `modules/amazonmq/`

Recursos requeridos

- `aws_mq_broker`
  - Engine: `RABBITMQ`

- Engine version: `3.11.20`
- Deployment mode: `SINGLE_INSTANCE`
- Instance type: `mq.m5.large`
- Subnet: primera subnet privada
- Security groups: `sg_rabbitmq`
- `publicly_accessible = false`
- Usuario y password como variables sensibles

Variables sensibles

```hcl
variable "rabbitmq_username" { type = string, sensitive = true }
variable "rabbitmq_password" { type = string, sensitive = true }
```

Outputs

```hcl
output "broker_id"      { value = aws_mq_broker.rabbitmq.id }
output "amqp_endpoint"  { value = tolist(aws_mq_broker.rabbitmq.instances)[0].endpoi
output "console_url"    { value = "https://${tolist(aws_mq_broker.rabbitmq.instances
```

---

9. Módulo: RDS PostgreSQL (Opción A — Recomendada)

Archivo: `modules/rds/`

Recursos requeridos

- `aws_db_subnet_group` — Con las subnets privadas
- `aws_db_instance`
  - Engine: `postgres`, versión `15`
  - Instance class: `db.t3.micro`
  - Storage: `20 GB`, tipo `gp2`
  - DB name: `tasksdb`
  - Multi-AZ: `false` (variable para habilitar en producción)
  - `skip_final_snapshot = true` (dev), `true` en producción cambiar a `false`

- `backup_retention_period = 7`
- Security group: `sg_rds`
- `publicly_accessible = false`

Variables sensibles

```hcl
variable "db_username" { type = string, sensitive = true }
variable "db_password" { type = string, sensitive = true }
```

Outputs

```hcl
output "db_endpoint" { value = aws_db_instance.postgres.address }
output "db_port"     { value = aws_db_instance.postgres.port }
output "db_name"     { value = aws_db_instance.postgres.db_name }
```

Esquema de base de datos (referencia, no gestionado por Terraform)

```sql
CREATE TABLE tasks (
  task_id   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  status    VARCHAR(50) NOT NULL,
  date      TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE orders (
  order_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  details     TEXT,
  created_at  TIMESTAMP NOT NULL DEFAULT NOW()
);
```

---

10. Módulo: EC2 PostgreSQL (Opción B — Alternativa)

Archivo: `modules/ec2-postgres/`

Recursos requeridos

- `data "aws_ami"` — Amazon Linux 2023, más reciente, owner `amazon`

- `aws_instance`
  - Tipo: `t3.micro`
  - Subnet: primera subnet privada
  - Security group: `sg_rds`
  - EBS root: `20 GB`, tipo `gp3`
  - `user_data` script que:
    1. Instala Docker (`yum install -y docker && systemctl enable docker && systemctl start docker`)
    2. Ejecuta: `docker run -d --name postgres --restart always -e POSTGRES_DB=tasksdb -e POSTGRES_USER=<var> -e POSTGRES_PASSWORD=<var> -p 5432:5432 postgres:15`

Outputs

```hcl
output "db_endpoint" { value = aws_instance.postgres.private_ip }
output "db_port"     { value = 5432 }
output "db_name"     { value = "tasksdb" }
```

---

11. Módulo: ECR

Archivo: `modules/ecr/`

Repositorios requeridos

Nombre del repo	Imagen que contiene
`api-rest`	API REST principal
`consumer-pool`	Worker consumidor general
`consumer-delete`	Worker consumidor de eliminación
`update-task`	Worker actualización de tareas

```hcl
image_tag_mutability = "MUTABLE"
scan_on_push         = true

lifecycle_policy = jsonencode({
  rules = [{
    rulePriority = 1
    selection    = { tagStatus = "any", countType = "imageCountMoreThan", countNumbe
    action       = { type = "expire" }
  }]
})
```

Outputs

```hcl
output "api_rest_repo_url"        { value = aws_ecr_repository.api_rest.repository_u
output "consumer_pool_repo_url"   { value = aws_ecr_repository.consumer_pool.reposit
output "consumer_delete_repo_url" { value = aws_ecr_repository.consumer_delete.repos
output "update_task_repo_url"     { value = aws_ecr_repository.update_task.repositor
```

---

12. Módulo: ECS

Archivo: `modules/ecs/`

Recursos base

- `aws_ecs_cluster` — Nombre: `tasks-cluster`
- `aws_iam_role` — ECS Task Execution Role con políticas:
  - `AmazonECSTaskExecutionRolePolicy`
  - `AmazonEC2ContainerRegistryReadOnly`
  - `CloudWatchLogsFullAccess`
- `aws_cloudwatch_log_group` — `/ecs/api-rest` y `/ecs/consumers`, retention `7 días`

Campo	Valor
Task CPU	`256`
Task Memory	`512`
Container name	`api-rest`
Image	`<account_id>.dkr.ecr.us-east-1.amazonaws.com/api-rest:latest`
Port	`8080`
Desired count	`1`
Subnets	Privadas
Security group	`sg_api`
Assign public IP	`false`
Load balancer	Registrado en `target_group_arn` del ALB

Variables de entorno del container API:

```
DB_HOST         = <db_endpoint>
DB_PORT         = 5432
DB_NAME         = tasksdb
DB_USER         = <var>
DB_PASSWORD     = <var>
RABBITMQ_URL    = <amqp_endpoint>
PORT            = 8080
```

Servicio: Consumer Pool

Campo	Valor
Container name	`consumer-pool`
Image	`<account_id>.dkr.ecr.us-east-1.amazonaws.com/consumer-pool:latest`

Campo	Valor
Desired count	`2`
Queue	`QUEUE_NAME=tasks_queue`
Subnets	Privadas
Security group	`sg_consumers`
Load balancer	Ninguno

Servicio: Consumer Delete

Campo	Valor
Container name	`consumer-delete`
Image	`<account_id>.dkr.ecr.us-east-1.amazonaws.com/consumer-delete:latest`
Desired count	`1`
Queue	`QUEUE_NAME=delete_queue`
Subnets	Privadas
Security group	`sg_consumers`

Servicio: Update Task

Campo	Valor
Container name	`update-task`
Image	`<account_id>.dkr.ecr.us-east-1.amazonaws.com/update-task:latest`
Desired count	`1`
Queue	`QUEUE_NAME=update_queue`
Subnets	Privadas
Security group	`sg_consumers`

Variables de entorno comunes a todos los consumers:

```
DB_HOST        = <db_endpoint>
DB_PORT        = 5432
DB_NAME        = tasksdb
DB_USER        = <var>
DB_PASSWORD    = <var>
RABBITMQ_URL   = <amqp_endpoint>
```

---

13. Root Module (environments/dev/)

main.tf — Orden de instanciación de módulos

```hcl
module "networking"       { source = "../../modules/networking" ... }
module "security_groups"  { source = "../../modules/security-groups", vpc_id = modul
module "alb"              { source = "../../modules/alb", subnets = module.networkin
module "amazonmq"         { source = "../../modules/amazonmq", subnets = module.netw
module "rds"              { source = "../../modules/rds", subnets = module.networkin
module "ecr"              { source = "../../modules/ecr" }
module "ecs"              { source = "../../modules/ecs", ... todos los outputs ante
```

variables.tf

```hcl
variable "aws_region"          { type = string, default = "us-east-1" }
variable "environment"         { type = string, default = "dev" }
variable "db_username"         { type = string, sensitive = true }
variable "db_password"         { type = string, sensitive = true }
variable "rabbitmq_username"   { type = string, sensitive = true }
variable "rabbitmq_password"   { type = string, sensitive = true }
```

outputs.tf

```hcl
```

```
output "alb_dns_name"          { value = module.alb.alb_dns_name }
output "rabbitmq_console_url"  { value = module.amazonmq.console_url }
output "db_endpoint"           { value = module.rds.db_endpoint }
output "ecr_urls"              { value = module.ecr }
```

terraform.tfvars (sin secrets)

```hcl
aws_region  = "us-east-1"
environment = "dev"
# db_username, db_password, rabbitmq_username, rabbitmq_password
# → Pasar como variables de entorno:
# export TF_VAR_db_password="..."
# export TF_VAR_rabbitmq_password="..."
```

---

14. Providers y Backend

providers.tf:

```hcl
terraform {
  required_version = ">= 1.5"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
  backend "local" {
    path = "terraform.tfstate"
  }
}

provider "aws" {
  region = var.aws_region
}
```

> **Nota:** El backend `local` es adecuado para el entorno Docker de desarrollo. Para equipos o CI/CD, migrar a `backend "s3"` con DynamoDB para locking.

15. Comandos de Despliegue (desde el container Docker)

```bash
# 1. Verificar credenciales
aws sts get-caller-identity

# 2. Navegar al entorno
cd /infra/environments/dev

# 3. Exportar secrets como variables de entorno (NO escribirlos en archivos)
export TF_VAR_db_username="admin"
export TF_VAR_db_password="<password_seguro>"
export TF_VAR_rabbitmq_username="admin"
export TF_VAR_rabbitmq_password="<password_seguro>"

# 4. Inicializar
terraform init

# 5. Validar sintaxis
terraform validate

# 6. Formatear
terraform fmt -recursive

# 7. Plan
terraform plan -out=tfplan -var-file=terraform.tfvars

# 8. Aplicar
terraform apply tfplan

# 9. Ver outputs
terraform output
```

Push de imagen a ECR (desde el container Docker)

```bash
```

```bash
# Obtener el URL del repo desde outputs
ECR_URL=$(terraform output -raw ecr_urls | jq -r '.api_rest_repo_url')
AWS_ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)

# Login a ECR
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin ${AWS_ACCOUNT_ID}.dkr.ecr.us-east-1.a

# Build y push
docker build -t api-rest .
docker tag api-rest:latest ${ECR_URL}:latest
docker push ${ECR_URL}:latest
```

Destrucción de infraestructura

```bash
terraform destroy -var-file=terraform.tfvars
```

---

16. Variables de Entorno por Servicio (resumen)

Variable	API REST	Consumer Pool	Consumer Delete	Update Task
`DB_HOST`	✅	✅	✅	✅
`DB_PORT`	✅	✅	✅	✅
`DB_NAME`	✅	✅	✅	✅
`DB_USER`	✅	✅	✅	✅
`DB_PASSWORD`	✅	✅	✅	✅
`RABBITMQ_URL`	✅	✅	✅	✅
`PORT`	8080	—	—	—
`QUEUE_NAME`	—	`tasks_queue`	`delete_queue`	`update_queue`

---

17. Consideraciones de Seguridad

- Todos los recursos de cómputo y datos están en **subnets privadas**.
- Solo el ALB está en subnets públicas.
- Las credenciales (DB password, RabbitMQ password) **nunca** deben ir en archivos `.tf` o `.tfvars` — usar variables de entorno `TF_VAR_*` o AWS Secrets Manager.
- Los security groups aplican el principio de **mínimo privilegio**: cada servicio solo acepta tráfico de quien lo necesita.
- ECR tiene `scan_on_push = true` para detección de vulnerabilidades en imágenes.
- RDS con `backup_retention_period = 7` para recuperación ante fallos.

---

18. Limitaciones y Próximos Pasos (fuera de alcance v1)

Elemento	Estado	Notas
HTTPS / certificado SSL	Pendiente	Requiere dominio + ACM certificate
Auto Scaling ECS	Pendiente	Agregar `aws_appautoscaling_target`
Multi-AZ para RDS	Pendiente	Cambiar variable `multi_az = true`
Backend remoto (S3+DynamoDB)	Pendiente	Para trabajo en equipo o CI/CD
Secrets Manager	Pendiente	Reemplazar `TF_VAR_*` con `aws_secretsmanager`
CI/CD pipeline	Pendiente	GitHub Actions o CodePipeline para auto-deploy
Amazon MQ Multi-AZ	Pendiente	Cambiar a `ACTIVE_STANDBY_MULTI_AZ`

---

19. Glosario

Término	Descripción
ALB	Application Load Balancer — balanceador HTTP/HTTPS de AWS
ECS Fargate	Plataforma serverless de contenedores de AWS
Amazon MQ	Servicio gestionado de message broker (RabbitMQ/ActiveMQ)

Término	Descripción
AMQP	Protocolo de mensajería usado por RabbitMQ (puerto 5672)
ECR	Elastic Container Registry — registro privado de imágenes
RDS	Relational Database Service — base de datos gestionada AWS
NAT Gateway	Permite salida a internet desde subnets privadas
IGW	Internet Gateway — entrada/salida de internet en la VPC
sg	Security Group — firewall virtual de AWS
DXA	Unidad de medida interna de documentos Office

Fin del documento spec.md — v1.0